

QUANTUM SEED GENERATOR v1.5.1 — Systemdokumentation

Hardware-Entropie-basierter BIP39-Seed-Generator auf Raspberry Pi 5. Stand: 30. August 2026 (Skriptversion 1.5.1, nach vier externen Audit-Runden). Diese Dokumentation beschreibt Zweck, Aufbau, Kryptographie und Sicherheitsarchitektur des Systems.

1. Zweck und Funktionsbeschreibung

Der QUANTUM SEED GENERATOR erzeugt BIP39-Seed-Phrasen (12 oder 24 Wörter) für Bitcoin-Wallets aus vier voneinander unabhängigen physikalischen Entropiequellen. Das System läuft vollständig offline auf einem Raspberry Pi 5, verarbeitet alle Geheimnisse ausschließlich im Arbeitsspeicher und hinterlässt keine Spuren auf Datenträgern. Die erzeugte Wortfolge ist standardkonform und kann in jede BIP39-kompatible Wallet importiert werden — Hardware-Wallets wie ColdCard, Trezor, Ledger, OneKey, BitBox ebenso wie Software-Wallets wie Electrum, Sparrow, Trust Wallet, BlueWallet u. v. a.: Der Standard garantiert, dass wirklich jede kompatible Wallet dieselben Schlüssel ableitet.

- Vier Entropiequellen: radioaktiver Zerfall, IR-Sensorrauschen, Hardware-RNG, Würfel
- Zweisprachige Programmführung (Deutsch/Englisch) mit Standard- und Transparenz-Modus
- Vollständige Nachvollziehbarkeit: im Transparenz-Modus sind alle Rohdaten und Hashes live sichtbar
- Erzwungene Offline-Umgebung: WLAN/Bluetooth-Abschaltung (rfkill + Firmware), Ethernet-Kabelprüfung
- Hardware-Healthcheck, statistische Zerfallsprüfung, Papier-Verifikation der Abschrift
- Spurenfreier Abschluss: Bildschirm-/Scrollbar-Löschung, optionale History-Bereinigung

2. Stärken gegenüber der Seed-Erzeugung in Standard-Hardware-Wallets

Handelsübliche Hardware-Wallets erzeugen den Seed aus einem einzigen, proprietären Chip-Zufallsgenerator, dessen Funktionsweise der Nutzer weder einsehen noch prüfen kann. Dieses System verfolgt den entgegengesetzten Ansatz:

Aspekt	Hardware-Wallet	Dieses System
Entropiequellen	1 (Chip-RNG, Blackbox)	4 unabhängige, davon 1 quantenphysikalisch (Zerfall) und 1 hardwareunabhängig (Würfel)
Prüfbarkeit	keine (Closed Source/Silizium)	vollständig: Open-Source-Skript, Transparenz-Modus zeigt jeden Zwischenwert
RNG-Backdoor-Risiko	Totalausfall der Sicherheit	wirkungslos: HMAC-Kombinierer mit Würfeln als unabhängigem Schlüssel
Entropie-Nachweis	Herstellerangabe	konservativ bilanziert und erzwungen (mind. 2× Ziel)
Lieferketten-Angriff	Gerät könnte manipuliert ankommen	Standardkomponenten, selbst zusammengestellt, Verhalten messbar
Seed-Anzeige-Umgebung	Gerätedisplay	erzwungene Offline-Prüfung unmittelbar vor Anzeige (Manipulationserkennung)

Wichtig zur Einordnung: Eine hochwertige Hardware-Wallet ist für die Aufbewahrung und Signierung weiterhin sinnvoll. Dieses System ersetzt nur den Schritt der Seed-ERZEUGUNG durch einen transparenten, mehrquelligen Prozess — der fertige Seed kann anschließend in eine Hardware-Wallet importiert werden.

3. Technische Komponenten

Komponente	Typ / Modell	Aufgabe
Einplatinencomputer	Raspberry Pi 5 (BCM2712), Raspberry Pi OS	Ausführung, Messwerterfassung, Kryptographie
Geigerzähler	RadiationD-v1.1 (CAJOE) mit J315-Zählrohr	Quantenentropie: Zeitpunkte radioaktiver Zerfälle (Hintergrundstrahlung)
Wärmebildsensor	MLX90640 (32×24 Thermopile, I2C 0x33)	Sensorrauschen: unkalibrierte ADC-Rohframes
Hardware-RNG	BCM2712 TRNG (/dev/hwrng)	5 × 512 Bit in Einzelproben mit thermischer Anregung dazwischen; Qualitätsprüfung pro Probe
Würfel	Präzisionswürfel (kasinotauglich empfohlen)	hardwareunabhängige letzte Quelle, HMAC-Schlüssel
Widerstand	10 kΩ (Signalleitung CAJOE)	Strombegrenzung/Schutz des GPIO-Eingangs
Software	btc_seedgen.py + install_seedgen.py (Python 3, lgpio, smbus2)	Erfassung, Prüfungen, Kombination, BIP39-Kodierung

4. Technische Verdrahtung

Alle Angaben beziehen sich auf den 40-Pin-Header des Raspberry Pi 5 (physische Pin-Nummern; Orientierung: USB/Ethernet nach unten, Pin 1 oben links, Ecke nächst SD-Karte). Der vollständige Plan mit allen 40 Pins ist im Installationsskript abrufbar: `python3 install_seedgen.py --pinout`

MLX90640 IR-Kamera

Kamera-Leitung	Pi-Pin	Funktion
VCC (gelb)	Pin 1	3,3 V — niemals 5 V!
SDA (violett)	Pin 3	GPIO2 / I2C1 SDA
SCL (grün)	Pin 5	GPIO3 / I2C1 SCL
GND (blau)	Pin 6	Masse

CAJOE-Geigerzähler

Verbindung	Pi-Pin	Hinweis
5-V-Versorgung	Pin 2 (oder eigenes USB-Netzteil)	Board-Versorgung
GND	Pin 9	gemeinsame Masse zwingend
Signal (weiß)	Pin 11 (GPIO17)	Abgriff an P3-VIN, 10 k Ω in Reihe; entscheidend: GPIO17 biasfrei (Pull None) — vom Installationsskript dauerhaft gesetzt (<code>gpio=17=ip,pn</code>)

5. Kryptographie des Seed-Erzeugungsprozesses

5.1 Abstrakte Erklärung

Man kann sich den Prozess wie das Anrühren einer unfälschbaren Farbe vorstellen: Vier Personen, die einander nicht kennen, geben je einen eigenen, geheimen Farbtön in einen Mixer (Zerfalls-Timing, Kameraauschen, Chip-Zufall, Würfel). Der Mixer (Hash-Funktion) verrührt alles so gründlich, dass aus der Mischfarbe keiner der Ausgangstöne rekonstruierbar ist — und schon ein einziges anderes Farbmolekül eine komplett andere Mischfarbe ergäbe. Entscheidend: Selbst wenn drei der vier Personen heimlich zusammenarbeiten und ihre Töne absprechen, bleibt die Mischfarbe unvorhersagbar, solange die vierte ehrlich ist. Die Würfel spielen dabei eine Sonderrolle: Sie sind der 'Schlüssel zum Mixer' — eine Quelle, die kein Chiphersteller, keine Firmware und kein Lieferant je beeinflussen konnte, weil sie aus der Hand des Nutzers kommt.

5.2 Technische Erklärung — Schritt für Schritt

Schritt 1 — Erfassung der Rohdaten. Jede Quelle liefert vollständige Rohdaten: der HWRNG 5 × 64 Bytes in Einzelproben (mit thermischer CPU-Anregung zwischen den Proben); der Geigerzähler die Kernel-Zeitstempel aller Zerfallsereignisse in Nanosekunden (je 8 Bytes, Interrupt-genau); die Kamera komplette unkalibrierte RAM-Frames (832 16-Bit-Worte = 1664 Bytes pro Frame, nur Frames mit ≥ 85 % Pixeländerung gegenüber dem letzten akzeptierten Frame); die Würfel die vollständige Wurfserie als Basis-6-Zahl.

Schritt 2 — Domain-separiertes Hashing pro Quelle. Jede Quelle wird einzeln durch SHA-512 verdichtet. Ein Domänen-Präfix und ein Längensfeld verhindern, dass Daten einer Quelle als andere Quelle interpretiert werden können (Format-Eindeutigkeit):

```
H_x  = SHA-512( "BTC-SEEDGEN-v1" || tag_x || len64(daten_x) || daten_x )
tags : hwrng, decay, cam, dice, os
```

Schritt 3 — Hardware-Pool. Die drei Hardware-Digests werden zusammen mit einem zusätzlichen Betriebssystem-Beitrag (os.urandom(64), Defense-in-Depth) erneut gehasht:

```
H_os  = SHA-512( DOM || "os"      || len || os.urandom(64) )
POOL  = SHA-512( DOM || "pool"    || H_hw || H_dec || H_cam || H_os )
```

Schritt 4 — HMAC-Kombination mit den Würfeln als Schlüssel. Der Kern des Schemas: HMAC-SHA512 ist ein beweisbar robuster Zwei-Quellen-Kombinierer. Das Ergebnis ist unvorhersagbar, solange MINDESTENS EINE Seite (Schlüssel oder Nachricht) unvorhersagbar ist. Damit retten die Würfel den Seed selbst bei vollständig kompromittierter Hardware — und umgekehrt:

```
FINAL = HMAC-SHA512( key = H_dice , msg = POOL )           # 512 Bit
```

Schritt 5 — Entropie-Extraktion und BIP39-Kodierung. Für 24 Wörter werden die ersten 256 Bit von FINAL entnommen (Präfix einer PRF-Ausgabe ist selbst PRF-Ausgabe — verlustfrei sicher, gleiches Prinzip wie SHA-512/256). Daran wird die BIP39-Checksumme angehängt (erste ENT/32 = 8 Bit von SHA-256(ENTROPY)); die 264 Bit werden in 24 Gruppen à 11 Bit zerlegt, jede Gruppe indiziert ein Wort der offiziellen, per SHA-256 verifizierten Wortliste (2048 Wörter).

Schritt 6 — Wallet-Seite (außerhalb dieses Systems). Beim Import führt die Wallet standardkonform PBKDF2-HMAC-SHA512 mit 2048 Iterationen aus (Mnemonic + optionale Passphrase) und leitet daraus per BIP32/BIP84 die Schlüssel ab. Iterationen im Generator selbst wären Sicherheitstheater: Key-Stretching hilft nur schwachen Geheimnissen — 256 Bit echte Entropie brauchen keine Streckung.

5.3 Die Entropiequellen und ihr Beitrag zur Gesamtsicherheit

Quelle	Physikalischer Ursprung	Konservative Kreditierung	Besonderer Wert
Radioaktiver Zerfall	Quantenmechanik: Zerfallszeitpunkt prinzipiell unvorhersagbar	2 Bit/Ereignis, mind. 128 Ereignisse (= 256 Bit)	einzige Quelle mit fundamentaler (nicht nur praktischer) Unvorhersagbarkeit; Poisson-Statistik prüfbar

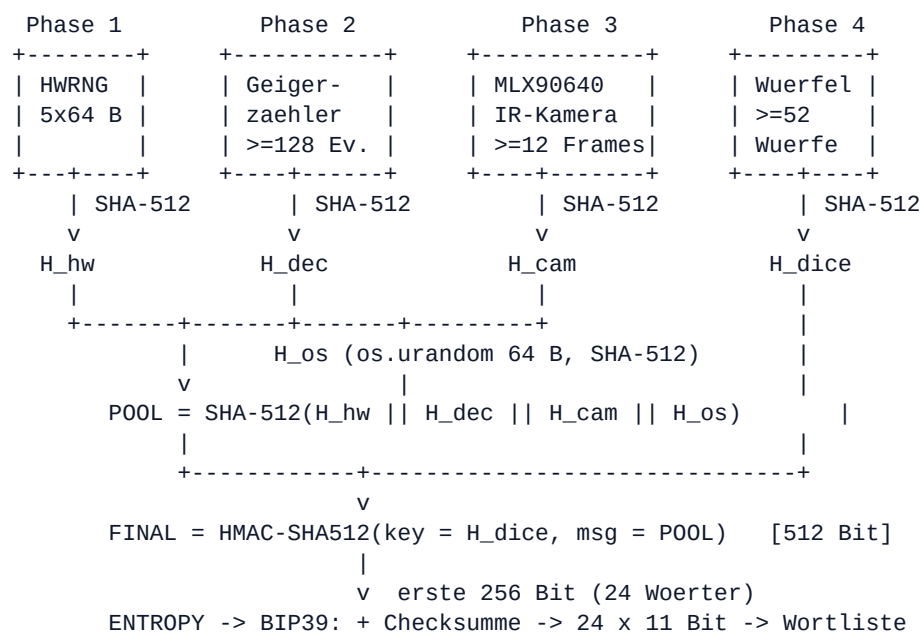
Quelle	Physikalischer Ursprung	Konservative Kreditierung	Besonderer Wert
IR-Sensorrauschen	thermisches Rauschen + ADC-Quantisierung über 768 Pixel	64 Bit/Frame, mind. 12 Frames; 85%-Filter erzwingt Frische	hohe Datenrate; Nutzerbewegung (Hand) koppelt makroskopische Unvorhersagbarkeit ein
BCM2712 HWRNG	Chip-interner TRNG	512 Bit roh, 256 kreditiert; Monobit-/Diversitätsprüfung	etablierte Grundversorgung; als EINE von vier Quellen unkritisch selbst bei Defekt
Würfel	Chaotische Mechanik in Nutzerhand	$\log_2(6) \approx 2,585$ Bit/Wurf, mind. 52 Würfe (≈ 134 Bit), 100 empfohlen (≈ 258 Bit)	hardware- und lieferkettenunabhängig; HMAC-Schlüssel = letzte Verteidigungslinie

Sicherheitsaussage des Kombinerers: Die Gesamtentropie des Seeds ist $\min(\text{Eingangsentropie}, 256 \text{ Bit})$, und ein Angreifer müsste ALLE vier Quellen gleichzeitig vorhersagen können. Die Bilanz erzwingt zusätzlich, dass die konservativ kreditierte Summe mindestens das Doppelte des Ziels beträgt.

5.4 Detaillierter Prozessablauf mit Illustrationen und Rechenbeispielen

Dieser Abschnitt geht jede Phase einzeln durch — mit Datenfluss-Diagrammen, konkreten Byte-Layouts und nachrechenbaren Zahlenbeispielen. Alle Beispielwerte wurden maschinell mit denselben Algorithmen berechnet, die auch das Skript verwendet.

5.4.1 Gesamt-Datenfluss



Lesart: Die vier Quellen laufen getrennt durch domänen-separierte Hashes. Drei Hardware-Quellen und der OS-Beitrag bilden den Pool (Nachricht), die Würfel bleiben bewusst außerhalb und werden erst als HMAC-Schlüssel eingekoppelt — dadurch genügt EINE ehrliche Seite für einen sicheren Seed.

5.4.2 Phase 1 — HWRNG: 5 Proben, thermische Anregung, Monobit-Beispiel

Aus `/dev/hwrng` werden fünf Proben à 64 Bytes gelesen — mit jeweils 2 Sekunden Abstand. In jeder Pause heizt ein zeitboxierter SHA-512-Burst die CPU auf: Die nächste Probe entsteht so unter veränderten physikalischen Bedingungen (Die-Temperatur, Versorgungslast) — der Jitter der TRNG-Ringoszillatoren ist temperatur- und spannungsabhängig. Das ist Dekorrelations-Verstärkung nach demselben Prinzip wie die Handbewegung vor der Kamera, ausdrücklich KEINE zusätzliche Entropiequelle. Es entschärft insbesondere die Schwächeklasse 'RNG direkt nach dem Kaltstart noch nicht warmgelaufen'.

Ablauf Phase 1:

```

Probe 1 (64 B) -> Pruefung -> 2 s Hash-Burst (heizt CPU)
Probe 2 (64 B) -> Pruefung -> 2 s Hash-Burst
...           -> ...       -> ...
  
```

Probe 5 (64 B) -> Pruefung -> Kreuzvergleich aller 5 Proben

Pruefungen PRO Probe:	Kreuzvergleich:
Diversitaet >= 32 Bytewerte	alle 5 Proben paarweise verschieden
Monobit 30-70 % Einsen	(haengender RNG fliegt sofort auf)

Jitter-Beiprodukt (unkreditiert, 0 Bit): pro Pause werden erreichte SHA-512-Iterationszahl + exakte Dauer in ns geerntet (Scheduler-/Cache-Jitter) und den Rohdaten angehaengt: $5 \times 64 \text{ B} + 4 \times 16 \text{ B} = 384 \text{ B}$ in den Hash. Kreditiert bleiben konservativ 256 Bit – die thermische Anregung ist Qualitaetsmarge, kein Bilanzposten.

Rechenbeispiel Monobit (je Probe, Beispiellauf):
512 Bit gelesen, davon Einsen gezaehlt: 259
Erwartung bei fairem RNG: 256 ($= 512 \times 0,5$)
Standardabweichung sigma: $\text{sqrt}(512 \times 0,5 \times 0,5) = 11,31$
Abweichung: $|259 - 256| = 3 = 0,27 \text{ sigma} \rightarrow \text{BESTANDEN}$
(Abbruchbereich erst bei grober Schiefelage, z. B. 340 Einsen = 7,4 sigma)

5.4.3 Phase 2 — Radioaktiver Zerfall: vom Impuls zum Byte

Jede Detektion eines Zerfallsprodukts erzeugt am P3-VIN-Abgriff einen Impuls; der Kernel liefert dazu einen monotonen Zeitstempel in Nanosekunden. Gehasht werden die vollständigen 8-Byte-Zeitstempel aller Ereignisse — nicht nur Differenzen oder einzelne Bits.

Beispiel (echte Messreihe, ~15 CPM):

Ereignis	Zeitstempel t [ns]	Delta zum Vorgaenger
1	35806374994711	--
2	35807625077266	1,250 s
3	35812116678703	4,492 s
4	35816465826823	4,349 s
5	35820107988436	3,642 s

Byte-Strom an SHA-512 (big-endian, 8 B je Ereignis):
00 00 20 91 5f 8e 3b 57 | 00 00 20 91 aa 03 76 12 | ...
Die Unvorhersagbarkeit steckt in den niederwertigen Bytes: selbst bei bekannter MITTLERER Rate ist der exakte Nanosekunden-Zeitpunkt jedes einzelnen Zerfalls quantenmechanisch offen.

Kreditiert werden konservativ nur 2 Bit pro Ereignis, obwohl die Nanosekunden-Zeitstempel real deutlich mehr tragen — Sicherheitsmarge nach unten. Für 24 Wörter (256-Bit-Ziel) sind daher mindestens 128 Ereignisse Pflicht: $128 \times 2 \text{ Bit} = 256 \text{ Bit}$ allein aus dieser Quelle.

Statistik-Wächter (Audit M4) – Rechenbeispiel mit 128 Intervallen:
Mittelwert $m = 4,0 \text{ s} \rightarrow \text{CPM} = 60/m = 15$
(1) Plausibilitaetsfenster: $1 \leq 15 \leq 3000$ BESTANDEN
(2) Variationskoeffizient: $\text{CV} = s/m = 3,8/4,0 = 0,95 \sim 1$ BESTANDEN
(Exponentialverteilung hat $\text{CV} = 1$; 50-Hz-Brumm mit Jitter haette z. B. $\text{CV} = 0,05 \rightarrow \text{SOFORTABBRUCH}$)
(3) Chi-Quadrat gegen Exponentialverteilung, 8 gleichwahrscheinliche Klassen, erwartet je 16:
beobachtet: 14 18 15 17 16 13 19 16
 $\chi^2 = (4+4+1+1+0+9+9+0)/16 = 1,75 < 24 \text{ (Grenze)}$ BESTANDEN

5.4.4 Phase 3 — IR-Kamera: Frame-Aufbau und 85%-Filter

Ein MLX90640-Rohframe (unkalibrierter RAM-Auszug):
 $832 \text{ Worte} \times 16 \text{ Bit} = 1664 \text{ Bytes}$
 $= 768 \text{ Pixel-ADC-Werte} (32 \times 24 \text{ Thermopile})$
 $+ 64 \text{ Kontroll-/Kalibrierworte}$

Beispiel-Pixelwerte (hex): fff2 0011 ffe9 002a 0007 ...
Die Entropie sitzt in den untersten Bits (thermisches Rauschen + ADC-Quantisierung) – gehasht wird trotzdem IMMER das komplette Frame.

85%-Filter (Frische-Zwang) – Beispiel:
Vergleich neues Frame vs. letztes AKZEPTIERTES Frame:
veraenderte Worte: 731 von 832 $\rightarrow \text{Diff} = 87,9 \% \geq 85 \% \text{ AKZEPTIERT}$

veraenderte Worte: 615 von 832 -> Diff = 73,9 % < 85 % VERWORFEN

Der Filter erzwingt, dass jedes kreditierte Frame substantiell neue Information trägt (Nutzer bewegt die Hand vor der Optik). Ein getrennter Wächter erkennt eingefrorene Sensorik (200 Frames < 5 % Differenz), und ein 5-Minuten-Geduldsfenster gibt dem Menschen Zeit, ohne bei kurzem Zögern abzubrechen. Kreditierung: konservativ 64 Bit pro akzeptiertem Frame; Minimum 12 Frames = 768 Bit.

5.4.5 Phase 4 — Würfel: Basis-6-Kodierung Schritt für Schritt

Die Wurffolge wird als EINE große Zahl im Stellenwertsystem zur Basis 6 kodiert (Augenzahl minus 1 je Stelle) und zusammen mit der Wurffanzahl byteweise ghasht — verlustfrei und eindeutig:

```
Beispiel mit 4 Wuerfen: 4, 1, 5, 2
Ziffern (Auge-1):      3, 0, 4, 1
Wert = ((3*6 + 0)*6 + 4)*6 + 1
      = (18*6 + 4)*6 + 1 = 112*6 + 1 = 673
Byte-Kodierung: [Anzahl als 4 B] [Wert big-endian]
                = 00 00 00 04 | 02 a1                (0x02a1 = 673)
```

```
Entropie pro Wurf: log2(6) = 2,585 Bit
52 Wuerfe -> 52 x 2,585 = 134,4 Bit    (erzwungenes Minimum, > 128!)
100 Wuerfe -> 100 x 2,585 = 258,5 Bit  (Empfehlung: volle 256-Bit-
                                         Wuerfelabdeckung im FULL-Modus)
110 Wuerfe -> 110 x 2,585 = 284,4 Bit  (LIGHT-Minimum: + 256 Bit HWRNG
                                         = 540 Bit >= 2 x 256 mit Marge)
```

5.4.6 Domain-Separation: das Byte-Layout der Hash-Eingaben

Jede Quelle wird exakt so an SHA-512 uebergeben:

```
+-----+-----+-----+-----+
| "BTC-SEEDGEN-v1" | tag    | len(daten) 8 B | daten      |
| 14 B Domaene    | z.B.    | big-endian      | variabel     |
|                  | "decay" |                  |            |
+-----+-----+-----+-----+
```

Warum? Ohne Praefixe koennte ein Byte-Strom aus Quelle A als Quelle B interpretiert werden (Kollision der Formate). Mit Domaene + Tag + Laenge ist jede Eingabe global eindeutig – auch gegenueber anderen Programmen, die SHA-512 verwenden.

5.4.7 Der HMAC-Kombinierer im Inneren

$HMAC\text{-}SHA512(K, m) = H((K' \text{ xor opad}) || H((K' \text{ xor ipad}) || m))$

```

          K = H_dice (64 B)          m = POOL (64 B)
              |                          |
          +-----+-----+          +-----+
          v                          v
K' xor ipad --> [ innerer Hash H( ipad-Key || POOL ) ]
K' xor opad  --> [ aeusserer Hash H( opad-Key || innerer ) ] -> FINAL
```

Sicherheitseigenschaft (Zwei-Quellen-Kombinierer):
unvorhersagbarer SCHLUESSEL + bekannte Nachricht -> FINAL unvorhersagbar
bekannter Schluessel + unvorhersagbare NACHRICHT -> ebenfalls
-> Ein Angreifer muesste Wuerfel UND alle Hardware-Quellen beherrschen.

Genau deshalb wäre einfaches XOR die schwächere Wahl: XOR ist zwar auch ein Kombinierer, aber anfällig, wenn eine Quelle die andere adaptiv beobachten könnte; die Hash-/HMAC-Konstruktion vernichtet jede Struktur der Eingaben und ist gegen solche Abhängigkeiten robust.

5.4.8 BIP39-Kodierung: vollständig durchgerechnetes Beispiel

Zur Nachvollziehbarkeit hier der offizielle BIP39-Testvektor mit 128-Bit-Entropie aus lauter Nullbytes (im echten Lauf steht hier ENTROPY = FINAL[:32]; das Rechenschema ist identisch, bei 24 Wörtern mit 256+8 Bit und 8-Bit-Checksumme):

Schritt 1 – Entropie:

ENT (128 Bit) = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Schritt 2 – Checksumme:

SHA-256(ENT) = 374708ffff7719dd5979ec875d56cd228...

erstes Byte = 0x37 = 00110111b

CS = erste ENT/32 = 4 Bit = 0011b = 3

Schritt 3 – Konkatenation ENT || CS (132 Bit) und Teilung in 12 x 11 Bit:

[000000000000] [000000000000] ... (11 Gruppen) ... [000000000011]

Indizes: 0 0 0 0 0 0 0 0 0 0 0 3

Schritt 4 – Wortliste (Zeile = Index + 1):

Index 0 -> abandon (x11), Index 3 -> about

Mnemonic: abandon abandon abandon abandon abandon abandon

abandon abandon abandon abandon abandon about

= offizieller BIP39-Testvektor -> Rechenweg verifiziert

Der letzte 11-Bit-Block enthält die 4 Checksummen-Bits als niederwertige Bits — deshalb endet der Nullvektor nicht auf 'abandon', sondern auf 'about' (Index 3). Jede BIP39-Wallet weltweit führt exakt dieselbe Rechnung aus und erkennt Tippfehler über diese Checksumme (bei 24 Wörtern werden 8 von 264 Bit geprüft; grobe Fehler fallen zusätzlich der Wortlisten-Suche auf).

5.4.9 Wallet-Seite: von der Wortfolge zu den Adressen

Mnemonic (+ optionale Passphrase)

| PBKDF2-HMAC-SHA512, 2048 Iterationen, Salt = "mnemonic"+Pass

v

512-Bit-Wallet-Seed

| BIP32: HMAC-SHA512("Bitcoin seed", Seed) -> Master-Key + Chaincode

v

m / 84' / 0' / 0' / 0 / i (BIP84, Native SegWit)

| | | | | +- Adress-Index i = 0,1,2, ...

| | | | | +--- extern (0) / Wechselgeld (1)

| | | +----- Konto 0

| | +----- Coin: Bitcoin (0)

| +----- Zweck: BIP84

v

Private Keys -> Public Keys (secp256k1) -> bc1q...-Adressen

Import-Referenz Electrum (offline): Options -> BIP39, Pfad m/84'/0'/0'

6. Sicherheitsmechanismen des Python-Skripts

Vor dem Start

- Sicherheitsregel-Box mit Pflichtbestätigung (Offline-Regeln, Smart-TV-Verbot, keine Handys/Kameras/Dritte im Raum)
- Root-Prüfung mit klarer sudo-Anleitung; Abbruch bei `python3 -O` (Selbsttests würden sonst unvollständig laufen)
- Remote-/Mitschnitt-Erkennung: SSH, tmux/screen, script(1), VNC/xrdp, Web-Terminals (ttyd, wetty, cockpit u. a.) und pty-Rekorder — sudo-robust über die /proc-Prozess-Ahnenkette, nicht nur über Umgebungsvariablen; zusätzlich stdout-TTY-Pflicht (Pipe/tee/Umleitung würden den Seed dauerhaft speichern → Abbruch)
- Erzwungene Funk-Abschaltung: `rkill` sofort + `dtoverlay`-Einträge (Firmware-Ebene, ab Boot) — mit Verifikation
- Ethernet-Kabel-Pflichtprüfung: Link-Erkennung, Zieh-Aufforderung, technische Nachmessung der Bestätigung
- Swap-Deaktivierung und RAM-Härtung: `mlockall()` pinnt alle Speicherseiten im RAM (greift auch, wenn `swapoff` scheitert), `PR_SET_DUMPABLE=0` blockiert `ptrace`-/proc-Auslesen zur Laufzeit, Core-Dumps deaktiviert (`RLIMIT_CORE=0`), Schutz gegen interaktiven Interpreter-Modus (`-i/PYTHONINSPECT`)
- Kryptographischer Selbsttest vor jedem Lauf: offizielle BIP39-Vektoren, Known-Answer-Vektor der kompletten Ableitungskette (fixes FINAL + Indizes) und Fault-Injection-Proben; Wortlisten-SHA-256-Verifikation; CLI-Parameter werden im Automatikmodus vor jedem systemverändernden Schritt validiert
- Hardware-Healthcheck: je 10 Live-Signale pro Quelle mit Anzeige, inkl. Messung der realen Zählrate für ehrliche Dauerprognosen

Während der Erzeugung

- Erzwungene Minima: ≥ 128 Zerfallsereignisse (24 Wörter), ≥ 12 Kamera-Frames, ≥ 52 Würfe; Entropie-Bilanz $\geq 2\times$ Ziel
- Statistische Zerfallsprüfung: CPM-Plausibilitätsfenster, Variationskoeffizient und Chi-Quadrat gegen die Exponentialverteilung — mit Totzeit-Korrektur (30-ms-Entprellung wird herausgerechnet, sonst Fehlalarme bei hohen Raten) und ratenadaptivem Timeout (bei langsamen Röhren wird nicht fälschlich abgebrochen); erkennt periodische Störsignale, die einfache Prüfungen täuschen würden
- Kamera-Frischefilter: nur Frames mit $\geq 85\%$ Pixeländerung; getrennter Frozen-Sensor-Wächter; 5-Minuten-Geduldsfenster für den Menschen
- Würfel-Gesundheitsprüfungen: Mindest-Diversität, Schlagseiten-Erkennung und Chi-Quadrat-Test über die sechs Augenzahlen (gezinkte Würfel und systematische Tippfehler fallen auf)
- Geigerzähler-Eingang: Entprellung (30 ms), automatische Flankenwahl und hardware-verifiziert biasfreier GPIO (Pull None — der hochohmige P3-VIN-Abgriff mit $10\text{ k}\Omega$ in Reihe würde von jedem internen Pull abgewürgt); per `--gpio-pull` umstellbar, mit 60-Sekunden-Auto-Fallback über alle Pull-Modi und Diagnosezeile (Chip, Pull, Ruhepegel, Flanke) beim Start

Bei Anzeige und Abschluss

- Finale Offline-Prüfung unmittelbar vor der Seed-Anzeige: WLAN, Bluetooth und Ethernet werden erneut gemessen; bei zwischenzeitlicher (Re-)Aktivierung Sicherheitsabbruch mit Manipulationswarnung — der Seed erscheint nie
- Mock-Modus: zufällig maskierte Wortindizes statt Wörtern, auch alle Transparenz-Geheimwerte (`H_dice`, `FINAL`, `ENTROPY`, Würfel-folge) maskiert — Testseeds sind nicht verwendbar; ein Mock-Lauf verändert das System nachweislich nicht (kein `rkill`, kein `swapoff`, keine `config.txt`-Einträge)
- Papier-Verifikation: rote Anleitung (Wörter MIT Nummern notieren — Reihenfolge!), Wiedereingabe NUR vom Papier, toleranter Parser, exakter Abgleich mit Positionsdiagnose
- Spurenfreier Abschluss: Bildschirm- und Scrollback-Löschung in terminalsicherer Reihenfolge (Home → 2J → 3J, mit `clear`-Fallback und Wiederholung), optionale Bash-History-Bereinigung
- Kein Geheimnis wird je auf einen Datenträger geschrieben: alle Geheimnisse existieren ausschließlich im RAM. Die einzigen Schreibzugriffe sind die angekündigten Härtungsschritte (`config.txt`-Overlays mit Backup und `fsync`, optionale `systemd`-Unit, Verlaufs-Bereinigung); Not-Löschung des Bildschirms auch bei `SIGTERM/SIGHUP/SIGQUIT` nach der Seed-Anzeige

Doppelberechnungs-Validierung gegen Bitflips (seit v1.1)

Der Raspberry Pi 5 hat keinen ECC-RAM — ein einzelnes gekipptes Bit zwischen Berechnung und Anzeige würde unbemerkt einen falschen Seed aufs Papier bringen. Dagegen rechnet das Skript die komplette Ableitungskette (Quell-Digests → POOL → HMAC-SHA512 → FINAL → Entropie-Kürzung → BIP39-Wortkodierung) zweimal vollständig und unabhängig aus denselben Eingaben (die OS-Nonce wird dafür genau einmal gezogen). Erst wenn FINAL, Indizes und fertige Wortfolgen beider Läufe übereinstimmen, geht es weiter. Zusätzlich: BIP39-Roundtrip (Wörter → Entropie → Prüfsumme → erneute Kodierung → Vergleich) und eine Wortlisten-Reverifikation direkt vor der Ausgabe — die im RAM liegende Liste wird identisch zum Ladeformat serialisiert und erneut gegen den SHA-256-Referenzhash geprüft, denn der Roundtrip allein wäre gegen eine im Speicher korruptierte Tabelle blind. Jede Abweichung führt zum lauten Abbruch mit eigenem Exit-Code 4 und ohne jede Teilanzeige.

LIGHT-Modus

Für Aufbauten ohne Geigerzähler und Kamera erzeugt der LIGHT-Modus Seeds aus HWRNG + Würfeln. Die Sicherheitslast trägt der HMAC-Kombinierer; erzwungenes Minimum sind 110 Würfe ($110 \times \log_2 6 + 256 \text{ Bit HWRNG} = 540 \text{ Bit} \geq 2 \times 256$ mit Marge). Ein Modus-Tag in der Pool-Berechnung trennt LIGHT- und FULL-Ableitungen kryptographisch (Domain-Separation) — identische Eingaben ergeben in beiden Modi nachweislich verschiedene Seeds.

CPM-Warndialog bei schwacher Zählrate

Liegt die im Healthcheck gemessene Zählrate unter 4 CPM, erklärt ein Dialog die Justage des Schwellen-Potentiometers auf der CAJOE-Platine (2–4 Umdrehungen gegen den Uhrzeigersinn, mit Hochspannungswarnung) und bietet Wiederholen oder bewusstes Fortfahren an — statt eines kommentarlosen, stundenlangen Laufs.

7. Theorie: Quantencomputer-Angriff — Hardware-Wallet-Seed vs. dieser 24-Wort-Seed

Ein zukünftiger fehlerkorrigierter Quantencomputer bedroht Bitcoin auf zwei getrennten Wegen, die man sauber auseinanderhalten muss:

Weg A — Shor-Algorithmus gegen die Elliptische-Kurven-Kryptographie (secp256k1): Sobald der öffentliche Schlüssel einer Adresse bekannt ist (spätestens beim ersten Ausgeben, bei wiederverwendeten Adressen dauerhaft), könnte Shor daraus den privaten Schlüssel in polynomieller Zeit berechnen. Ehrliche Feststellung: Gegen diesen Weg schützt KEIN Seed-Generator der Welt — weder eine Hardware-Wallet noch dieses System —, denn er greift die Signaturkurve an, nicht die Seed-Entropie. Die Verteidigung liegt auf Protokollebene (künftige Post-Quantum-Signaturen) und in der Praxis: Adressen nicht wiederverwenden, Guthaben nach Bekanntwerden realer Quantenfähigkeiten auf neue Verfahren migrieren.

Weg B — Grover-Algorithmus gegen die Seed-Entropie (Brute-Force): Grover beschleunigt unstrukturierte Suche quadratisch — die effektive Sicherheit einer n-Bit-Entropie sinkt auf $n/2$ Bit. Und genau hier trennt sich die Spreu vom Weizen:

Szenario	klassische Sicherheit	Quanten-Sicherheit (Grover)	Bewertung
24 Wörter, volle 256 Bit (dieses System, nachgewiesen)	2^{256}	2^{128}	dauerhaft sicher — 2^{128} gilt auch quantentechnisch als unerreichbar
24 Wörter aus intakter Hardware-Wallet	2^{256} (Herstellerangabe)	2^{128}	gleichwertig — SOFERN der RNG wirklich voll lieferte
24 Wörter aus fehlerhaftem/manipuliertem RNG (real dokumentierte Fälle: reduzierte effektive Entropie, z. B. 64–128 Bit)	2^{64} – 2^{128}	2^{32} – 2^{64}	gefährdet bis trivial brechbar — und der Nutzer kann es nicht erkennen
12 Wörter (128 Bit)	2^{128}	2^{64}	klassisch sicher, im Quantenzeitalter Grenzfall — daher hier 24 Wörter empfohlen

Fazit: Der Vorteil dieses Systems im Quantenkontext liegt nicht in einer magischen Quantenresistenz der Signaturen (die hat niemand), sondern in der GARANTIE der vollen 256 Bit Eingangsentropie. Grover halbiert Exponenten — wer mit nachweisbaren 256 Bit startet, behält 128 Bit Quantenmarge; wer unwissentlich mit einem geschwächten RNG startete, fällt unter Grover möglicherweise in brechbare Bereiche. Die Mehrquellen-Architektur mit Transparenz-Modus und erzwungener Bilanz macht genau diese stille Schwächung unmöglich. Zusätzlich gilt: Solange eine Adresse unbenutzt ist (Public Key nur als Hash bekannt), schützt auch gegen Weg A vorerst die Hash-Barriere — ein Grund mehr für Adress-Hygiene.

8. Betriebsablauf (Kurzreferenz)

- Einrichtung (einmalig, online erlaubt): `sudo python3 install_seedgen.py` → `sudo reboot`
- Start: `sudo python3 btc_seedgen.py` → Sprache wählen → Sicherheitsregeln bestätigen
- Automatik: Funk-Abschaltung, Ethernet-Kabel ziehen (erzwungen), Swap aus, Healthcheck mit Erfolgssignal
- Konfiguration: Modus (Standard/Transparenz), Wortzahl, Ereignisse/Frames/Würfe (Minima werden erzwungen; realistische Dauer aus gemessener Zählrate)
- Phasen 1–4: HWRNG → Zerfall → Kamera (Hand bewegen!) → Würfel eingeben
- Finale Offline-Prüfung → Seed-Anzeige → Wörter MIT Nummern auf Papier → Verifikation VOM PAPIER → Enter löscht Bildschirm+Scrollback → Terminal schließen
- Empfehlung für den Ernstfall (FULL): 24 Wörter, ≥ 512 Zerfallsereignisse, ≥ 100 Würfe; im LIGHT-Modus gilt das erzwungene Minimum von 110 Würfeln. Import-Verifikation auf zweitem Offline-Gerät (Electrum: Options → BIP39; Pfad `m/84'/0'/0'`)
- Diagnose bei Hardware-Problemen (tools/): `cajoe_test.py` misst die Zählrate, `gpio_decay_diagnose.py` findet die richtige Pull/Flanken-Kombination (testet alle neun, ändert nichts am System), `camera_test.py` und `camera_raw_test.py` prüfen die MLX90640; bei < 4 CPM führt das Hauptskript selbst durch die Poti-Justage

Exit-Codes des Skripts:

Code	Bedeutung
0	Erfolg — Seed erzeugt und Lauf sauber abgeschlossen
2	Sicherheits- oder Konfigurationsabbruch (Quelle ausgefallen, Statistik-Test, Parameterfehler, Nutzer lehnte Gate ab)
3	Finale Offline-Prüfung verletzt — Funk/Ethernet zwischenzeitlich (re-)aktiviert, Seed wurde nie angezeigt
4	Doppelberechnungs-Validierung fehlgeschlagen — Verdacht auf transienten Speicherfehler (Bitflip), Lauf wiederholen
130	Abbruch durch Benutzer (Strg+C)

9. Grenzen und Restrisiken (ehrliche Betrachtung)

- Shor-Risiko der Signaturkurve bleibt bestehen (siehe Kap. 7) — protokollseitig, nicht generatorseitig lösbar
- Python kann RAM nicht garantiert überschreiben (unveränderliche Objekte); Absicherung erfolgt über Swap-/Core-Dump-Sperre und Prozessende — für maximale Paranoia: Pi nach dem Lauf stromlos machen
- Healthchecks erkennen Defekte, keine perfekt getarnte Chip-Manipulation — dagegen steht die Architektur (Würfel-HMAC), nicht die Statistik
- Physische Sicherheit des Papier-Backups liegt außerhalb des Systems (Metallplatte, sichere Verwahrung, ggf. Passphrase als 25. Wort in der Wallet)
- Der Monitor ist Teil der Vertrauenskette (daher Smart-TV-Verbot in den Regeln)
- Ein einzelner Lauf ersetzt keine organisatorische Sicherheit: Testläufe mit Wegwerf-Seeds vor dem Ernstfall bleiben Pflicht
- Grenzen des Doppellaufs: deterministische Programmfehler treffen beide Läufe identisch (dagegen stehen die Known-Answer-Vektoren im Selbsttest); Bitflips in den Rohdaten VOR beiden Läufen sind unschädlich (verfälschter Zufall bleibt Zufall); ein Kippen erst im Anzeigepuffer fängt die BIP39-Prüfsumme bei der Test-Wiederherstellung — die Import-Verifikation vor der ersten Einzahlung bleibt deshalb Pflicht
- pty-Rekorder sind prinzipiell nicht vollständig erkennbar (z. B. als python-Prozess laufendes asciinema) — die Erkennung deckt die verbreiteten Werkzeuge ab, ersetzt aber nicht die Regel, die Zeremonie nur an einer frisch gestarteten, lokalen Konsole durchzuführen
- mlockall/PR_SET_DUMPABLE schützen nicht gegen root-Prozesse, Cold-Boot- oder DMA-Angriffe — die Kernempfehlung bleibt: ein frischer Boot pro Seed-Zeremonie, danach stromlos

10. Integrität und Reproduzierbarkeit

Das Repository liefert eine geschlossene Prüfsummenkette: sha256.txt deckt alle Projektdateien ab (Skripte, Dokumentation in docs/, Werkzeuge in tools/) — nach dem Klonen bzw. jeder Übertragung auf den Pi gilt: `sha256sum -c sha256.txt`, danach `sudo python3 btc_seedgen.py --selftest`. Der Selbsttest prüft die offiziellen BIP39-Vektoren, die Wortliste, die Doppelberechnungslogik per Fault-Injection und einen Known-Answer-Vektor der kompletten Ableitungskette (fest verdrahtetes FINAL und feste Indizes) — ein deterministischer Fehler im Kombinierer würde hier auffallen, bevor je ein Seed entsteht. Im Transparenz-Modus ist jeder Zwischenwert (H_hw, H_dec, H_cam, H_os, H_dice, POOL, FINAL, ENTROPY) angezeigt und mit Standardwerkzeugen unabhängig nachrechenbar — das System verlangt an keiner Stelle Vertrauen in unbelegte Behauptungen.

11. Glossar

Begriff	Bedeutung
BIP39	Standard zur Kodierung von Entropie als merkbare Wortfolge inkl. Checksumme
Entropie	Maß der Unvorhersagbarkeit; hier konservativ in Bit bilanziert
HMAC	Schlüsselabhängige Hash-Konstruktion; hier als robuster Zwei-Quellen-Kombinierer
Domain-Separation	Eindeutige Kennzeichnung der Hash-Eingaben, verhindert Format-Verwechslung
CPM	Counts per minute — Zählrate des Geigerzählers
Poisson-Prozess	Statistik unabhängiger Zufallseignisse; Kennzeichen echten Zerfalls ($CV \approx 1$, exponentielle Intervalle)
Grover/Shor	Quantenalgorithmen: quadratische Suche-Beschleunigung bzw. Brechen von Public-Key-Kryptographie
Air-Gap	Physische Trennung eines Systems von allen Netzwerken
KAT	Known-Answer-Test: fest verdrahtetes Soll-Ergebnis, gegen das der Selbsttest die Ableitungskette prüft
Doppelberechnung	Zweifache unabhängige Ausführung der Ableitungskette mit Vergleich vor der Anzeige (Bitflip-Erkennung)

12. Lizenz und Haftungsausschluss

Dieses Projekt (Quellcode, Installationsskript und diese Dokumentation) steht unter der MIT License — Copyright (c) 2026 Quantum Seed Generator Project. Jeder darf die Software kostenlos nutzen, prüfen, verändern und weiterverbreiten; einzige Bedingung ist die Beibehaltung des Lizenz- und Copyright-Hinweises. Der vollständige Lizenztext liegt in der Datei LICENSE des Repositories. Die Software wird 'WIE BESEHEN' ohne jede Gewährleistung bereitgestellt (AS IS, without warranty of any kind).

Dieses System ist ein Selbstbau-Projekt für technisch versierte Anwender. Die Verantwortung für Betrieb, Verwahrung des Backups und den Umgang mit Kryptowährungen liegt vollständig beim Nutzer. Vor der Verwendung mit echten Werten sind eigene Testläufe und die Import-Verifikation auf einem unabhängigen Offline-Gerät zwingend.